

Full Stack Development with MERN

Project Documentation Format

Date	19 July 2026
Team ID	XXXX XXXX XXXX
Project Name	Ucab – Smart Cab Booking Application
Maximum Marks	-

1. Introduction

Project Title: Ucab – Smart Cab Booking Application

Team Members: Hemanth (Full Stack Developer – client, server, database, deployment)

2. Project Overview

Purpose: Ucab digitizes the cab-hailing journey end-to-end — discovery, transparent fare estimation, live tracking, automatic payment and history — so riders can book a reliable cab in a few taps.

Features: user auth, nearby cab search with fare/ETA, map-based pickup/drop selection, live trip tracking, auto-pay, booking history, discount codes, donation option, in-ride refreshment purchase.

3. Architecture

Frontend: React.js single-page app with component-based structure (Auth, Booking, Tracking, Payment, History components), React Router for navigation, and Axios for API calls.

Backend: Node.js + Express.js REST API following the MVC pattern — routes define endpoints, controllers hold business logic (fare calculation, driver matching), and middleware handles auth (JWT) and validation.

Database: MongoDB with Mongoose schemas for User, Driver, Booking and Payment collections; relationships modeled via ObjectId references (e.g., Booking references User and Driver).

4. Setup Instructions

Prerequisites: Node.js (v18+), npm, MongoDB (local or Atlas connection string), Git.

Installation

- Clone the repository: `git clone https://github.com/Hemanth-1912/CAB-BOOKING.git`
- Install server dependencies: `cd server && npm install`
- Install client dependencies: `cd client && npm install`

- Create a .env file in /server with MONGO_URI, JWT_SECRET, and payment/maps API keys

5. Folder Structure

Client: /client/src contains components/ (booking, tracking, payment, history UI), pages/, services/ (API calls via Axios), and App.js for routing.

Server: /server contains routes/ (API endpoints), controllers/ (business logic), models/ (Mongoose schemas), middleware/ (auth & validation), and server.js as the entry point.

6. Running the Application

- Backend: npm start (inside /server) — runs Express server, default port 5000
- Frontend: npm start (inside /client) — runs React dev server, default port 3000

7. API Documentation

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate user and return JWT
GET	/api/cabs/nearby	Fetch nearby cabs with fare & ETA for given coordinates
POST	/api/bookings	Create a new booking
GET	/api/bookings/:userId	Fetch a user's booking history
POST	/api/payments/charge	Process automatic payment for a completed ride

8. Authentication

Authentication uses JSON Web Tokens (JWT): on login, the server issues a signed token containing the user ID and role; the client stores it and sends it in the Authorization header on protected requests; middleware verifies the token before allowing access to booking, payment, and history routes. Passwords are hashed with bcrypt before storage.

9. User Interface

Screens: Login/Signup, Booking (map-based pickup/drop selection with nearby cab list), Live Tracking, Payment confirmation, Booking History. [Insert UI screenshots/GIFs here from the CAB_BOOKING_VIDEOS folder.]

10. Testing

Manual functional and user acceptance testing was performed across auth, booking, tracking, payment, and history modules (see User Acceptance Testing FSD document in the Project Development Phase folder for the full test case log).

11. Screenshots or Demo

[Insert screenshots or a link to the demo video/recording stored in the CAB_BOOKING_VIDEOS folder of the repository.]

12. Known Issues

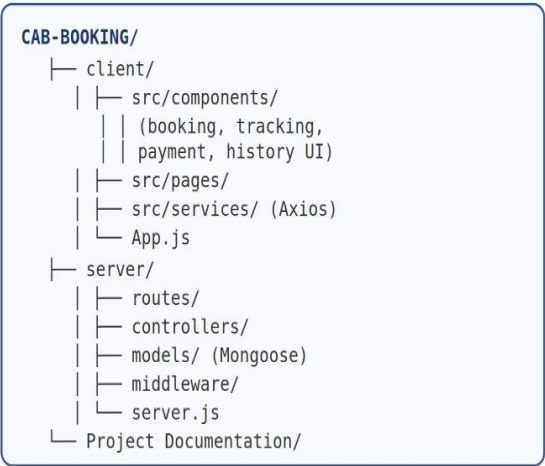
Live-tracking marker refresh can lag slightly (~2s) on poor network connections; being addressed by tuning the Socket.io emit interval.

13. Future Enhancements

- Driver-side mobile app with route optimization
- AI-based fare prediction and dynamic pricing
- In-app chat/call between rider and driver
- Loyalty points and referral discounts
- Multi-city and multi-language support

Full Stack Development Documentation — Ucab

Repository Structure



Documentation Sections

1. Introduction & 2. Project Overview
3. Architecture (Client / Server / DB)
4. Setup Instructions & 5. Folder Structure
6. Running the App & 7. API Docs
8. Authentication (JWT) & 9. UI
10. Testing & 11. Screenshots/Demo
12. Known Issues & 13. Future Enhancements

Core API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate & return JWT
GET	/api/cabs/nearby	Nearby cabs, fare & ETA
POST	/api/bookings	Create a new booking
POST	/api/payments/charge	Auto-charge saved payment method